Efficiency of Quick Sort:

Assume that the file size n (number of elements to be sorted) is a power of 2, say $n=2^m$ so that $m=\log_2 n$

Also assume that the proper position for the pivot always turns out to be the exact middle of the sub array. In that case there will be approximately n comparisons (actually n-1) on the first pass, after which the file is split into two subfile (two sub arrays) each of size n/2 approximately.

For each of these two files there are approximately n/2 comparisons and a total of four files each of size n/4 are formed. Each of these files requires n/4 comparisons yielding a total of n/8 subfiles.

After halving the subfiles m times, there are n files of size 1.

Thus the total number of comparisons for the entire sort is approximately,

n+ 2(n/2)+ 4(n/4)+ 8(n/8)+ ……..+ n(n/n)

⇨ n + n + n + n + …… + n (m times)

⇨ n x m

Thus total number of comparisons is O(n x m)

⇨ O (n log n)

Thus the quick sort is O(n log n) which is relatively efficient.